

Report on CPS Project (Technology - Boolean Satisfiability)

1. Overview

This report describes the variables, constraints, and overall modeling strategy used in the implementation of a solution to the Minimum Consistent Finite Automata problem using **Boolean Satisfiability** with **kissat** SAT solver. It attempts to find the smallest deterministic finite automaton (DFA) consistent with a given sample of accepted and rejected binary sequences.

2. Variables

The model defines the following key variables representing variable families:

- ***int id_T(int i, int b, int j)***: A transition variable which is true iff from state *i* with reading *b* goes to state *j*, mapping formula - $i \cdot 2^n + b \cdot n + j + 1$
- ***int id_A(int i)***: An accepting state indicator which is true iff state *i* is accepting state, mapping formula - $2^n \cdot n + i + 1$
- ***int id_S(int s, int p, int i)***: A state at prefix which is true iff DFA is in state *i* after reading *p* bits of sequence *s*, mapping formula - $2^n \cdot n + n + \text{static_cast<int>}(seq_offset[s] + p \cdot n + i + 1)$

Literals are then constructed with those functions:

- ***inline literal lit(int id)*** { return to_string(id) + " "; }
- ***inline literal T(int i, int b, int j)*** { return lit(id_T(i, b, j)); }
- ***inline literal A(int i)*** { return lit(id_A(i)); }
- ***inline literal S(int s, int p, int i)*** { return lit(id_S(s, p, i)); }

3. Functions

The program has some key functions:

- ***void add_clause(const clause& c)*** – used for adding clause to the CNF file
- ***void add_amo(const vector<literal>& z)*** - used for enforcing pairwise at-most-one constraint
- ***void add_exactly_one(const vector<literal>& z)*** – used for enforcing exactly-one constraint using at-least-one + pairwise at-most-one
- ***void write_CNF()***– used for writing all clauses for a fixed number of DFA states
- ***vector<vector<int>> read_sequences(istream& in)*** – reads the sequence of *n* bits and makes a *vector<int>* out of it, out of all sequences also makes and returns a *vector<vector<int>>*

4. Constraints

The following constraints are enforced in the model:

- 1) There is exactly **one state** per prefix – enforced by $add_exactly_one(S(s,p,i))$
- 2) **Empty** prefix (epsilon) is in the **state 0** – enforced by unit clause $add_clause(S(s,0,0))$;
- 3) There is only **one successor** as this is DFA – enforced by $add_exactly_one(T(i,b,j))$;
- 4) If state i is **followed** by state j with transition by 0/1 that transition must exist – enforced by propagation $add_clause(\neg S(s,p,i) \vee T(i,b,j) \vee S(s,p+1,j))$;
- 5) Finally state must be **acceptable** or **rejectable** – depending on label accept $\rightarrow add_clause(\neg S(s,m,i) \vee A(i))$; or reject $\rightarrow add_clause(\neg S(s,m,i) \vee \neg A(i))$;

5. Search Strategy

The number of states n is not a decision variable. Instead, the program performs a linear search from $n = 1$ up to the number of unique prefixes. For each n , a CNF file is generated and solved using Kissat. The smallest value of n for which the formula is satisfiable corresponds to the minimal consistent DFA.

6. Conclusion

- The model is designed to find the minimal number of states needed for a consistent DFA.
- It outputs the DFA as a list of transitions and acceptance flags for each state as well as provided input for corresponding output.